

Модульное тестирование с использованием Jest

describe(), test(), expect(), matchers, hooks, mocks и изоляция Service layer

Результаты занятия

- ✓ Устанавливать и запускать Jest в ESM-проекте Node.js
- ✓ Группировать tests через describe()
- ✓ Использовать test() и expect()
- ✓ Применять основные Jest matchers
- ✓ Проверять ошибки и async functions
- ✓ Использовать beforeEach() и afterEach()
- ✓ Создавать mock functions через jest.fn()
- ✓ Проверять вызовы dependency
- ✓ Изолировать Service от Prisma через dependency injection
- ✓ Запускать unit test suite и анализировать failures

Учебные вопросы

- 1 Jest в Node.js проекте** Установка, ESM-конфигурация и структура tests
- 2 test(), describe() и expect()** Организация test suite и matchers
- 3 Setup, teardown и async tests** beforeEach, afterEach, resolves и rejects
- 4 Mock functions** jest.fn(), calls, return values и spies
- 5 Unit testing Service layer** Dependency injection и изоляция Prisma



Учебный вопрос 1. Jest в Node.js проекте

Jest предоставляет test runner, assertions, mocks и инструменты организации test suite

Что такое Jest

- Jest — JavaScript testing framework, объединяющий test runner, assertions, mocking и инструменты отчётности.
- Test runner находит и запускает test files.
- expect() предоставляет assertions через matchers.
- jest.fn() создаёт mock functions.
- Hooks помогают готовить окружение перед tests.
- Для backend используем testEnvironment: node.

node:test и Jest

node:test

- Встроен в Node.js
- Не требует установки framework
- Подходит для базовых tests
- Использует node:assert

Jest

- Отдельный testing framework
- expect() и богатые matchers
- Mocks и spies из коробки
- Удобная организация крупных test suites

Установка Jest

BASH

```
npm install --save-dev jest
```

Jest в проекте

Исходный код и тесты → Jest → выполнение → отчёт о результатах

Jest запускает тесты, выполняет их в изолированной среде и показывает подробный отчёт: какие тесты прошли, а какие завершились с ошибкой.

1

Source code

Исходный код (Service)

```
src/
├── services/
│   └── JS productService.js
```

```
productService.js

export class ProductService {
  constructor(productRepo) {
    this.repo = productRepo;
  }

  async getById(id) {
    return this.repo.findById(id);
  }
}
```

Реализация сервиса, который мы тестируем.

2

Test files

Тестовые файлы (*.test.js)

```
tests/
└── JS productService.test.js
```

```
productService.test.js

import { ProductService } from
'../src/services/productService.js';

test('getById returns product', async () => {
  const repoMock = {
    findById: jest.fn().mockResolvedValue({
      id: 1,
      name: 'Laptop'
    })
  };

  const service = new ProductService(repoMock);
  const product = await service.getById(1);

  expect(product.name).toBe('Laptop');
  expect(repoMock.findById).toHaveBeenCalledWith(1);
});
```

Тесты проверяют поведение сервиса с использованием моков (jest.fn()).

3

Jest runner

Запуск и выполнение тестов



```
$ npx jest
```

- Находит тестовые файлы
- Выполняет тесты в изолированной среде
- Собирает результаты
- Показывает отчёт

4

Report

Отчёт о результатах

```
PS C:\project> npx jest

PASS tests/productService.test.js

✓ getById returns product (12 ms)
✓ create saves new product (8 ms)
✓ throws error when product not found (5 ms)

Test Suites: 1 passed, 1 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 0.893 s
Ran all test suites.
```

✓ **Все тесты пройдены!**
Код работает корректно.

✗ **Есть ошибки в тестах.**
Проверьте отчёт, исправьте код и запустите тесты снова.



Автоматизированные тесты с Jest помогают писать надёжный, поддерживаемый и качественный код.

Пиши тесты. Уверенно развивай проект.

ES Modules в нашем проекте

- Internet Shop использует "type": "module".
- Jest 30.5 всё ещё помечает native ESM support как experimental.
- Для запуска используется Node с --experimental-vm-modules.
- В ESM test files объект jest удобно импортировать из @jest/globals.
- Мы избегаем unstable module mocking там, где dependency injection проще и понятнее.

package.json для Jest + ESM

JSON

```
{
  "type": "module",
  "scripts": {
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js",
    "test:watch": "node --experimental-vm-modules node_modules/jest/bin/jest.js --watch"
  }
}
```

Что важно

- Jest запускается через Node.
- --experimental-vm-modules нужен для ESM support Jest.
- npm test запускает весь test suite.
- npm run test:watch включает watch mode.

jest.config.js

JAVASCRIPT

```
export default {  
  testEnvironment: "node",  
  transform: {},  
  clearMocks: true  
};
```

Структура tests

- Можно использовать каталог tests/.
- Файлы удобно называть *.test.js.
- Unit tests располагаем рядом по домену или в отдельном tests/unit.
- Название файла должно подсказывать, какой модуль проверяется.
- В лабораторной используем tests/unit/orderMath.test.js и productService.test.js.



Учебный вопрос 2. `test()`, `describe()` и `expect()`

Jest делает test suite читаемым и выразительным

Первый Jest test

JAVASCRIPT

```
import { test, expect } from "@jest/globals";
import { calculateOrderTotal } from "../../services/orderMath.js";

test("calculates total for several items", () => {
  const total = calculateOrderTotal([
    { price: 10000, quantity: 2 },
    { price: 5000, quantity: 1 }
  ]);

  expect(total).toBe(25000);
});
```

Что важно

- test() объявляет test case.
- expect(actual) создаёт assertion context.
- toBe(expected) проверяет строгое равенство примитивов.

describe()

- describe() группирует связанные test cases в логический test suite.
- Удобно группировать tests одной функции или Service.
- Вложенные describe позволяют разделять методы и сценарии.
- describe не является самим test case.
- Имена describe и test формируют читаемый отчёт.

Группировка через describe()

JAVASCRIPT

```
import { describe, test, expect } from "@jest/globals";

describe("calculateOrderTotal", () => {
  test("calculates total", () => {
    expect(calculateOrderTotal([
      { price: 1000, quantity: 2 }
    ])).toBe(2000);
  });
});
```

Структура теста в Jest

describe → test cases → expect → matcher → PASS/FAIL

Jest организует тесты в иерархию: тестовый набор (describe) содержит отдельные тесты (test/it).
Внутри каждого теста выполняются проверки с помощью expect и матчеров.



describe — группирует тесты,
test/it — отдельный тест,
expect — задаёт ожидаемое значение,
matcher — сравнивает и определяет результат.

productService.test.js

```
1 describe('ProductService', () => {  
  2 test('getById returns product', async () => {  
    const service = new ProductService(mockRepo);  
    const product = await service.getById(1);  
    3 expect(product).toBeDefined();  
    4 expect(product.name).toBe('Laptop');  
  });  
  test('throws error for not found product', async () => {  
    await expect(service.getById(999))  
      .rejects.toThrow('Product not found');  
  });  
});
```

1

describe

Тестовый набор (suite)

- Группирует связанные тесты
- Может содержать вложенные describe
- Имеет общий контекст (beforeAll, beforeEach и т.д.)



2

test / it

Тестовый случай (test case)

- Содержит конкретную проверку
- Может быть синхронным или асинхронным
- Имеет своё имя (описание)



3

expect

Функция для проверки

- Принимает фактическое значение
- Возвращает объект с матчерами
- Используется внутри теста



4

matcher

Матчер (метод сравнения)

- Сравнивает фактическое и ожидаемое значение
- Если условие истинно → тест проходит (PASS)
- Если ложно → тест падает (FAIL)

Иерархия тестов в Jest

Jest

describe('ProductService')

test('getById returns product')

test('throws error for not found product')

test('create saves product')

describe('Validation')

test('valid data')

test('invalid data')



PASS

Все ожидания выполнены.



FAIL

Хотя бы одно ожидание не выполнено.



Чёткая структура тестов делает код понятным, а результаты — надёжными.

Пиши тесты. Проверь поведение. Развивай проект.

Базовые Jest matchers

Matcher	Назначение	Пример
toBe()	Строгое равенство примитивов	<code>expect(total).toBe(25000)</code>
toEqual()	Сравнение объектов/массивов	<code>expect(product).toEqual({...})</code>
toBeTruthy()	Truthy value	<code>expect(result).toBeTruthy()</code>
toBeNull()	Значение null	<code>expect(product).toBeNull()</code>
toContain()	Содержит элемент	<code>expect(roles).toContain("admin")</code>
toHaveLength()	Длина массива/строки	<code>expect(items).toHaveLength(2)</code>

toBe() и toEqual()

toBe()

- Использует Object.is
- Подходит для number/string/boolean
- Проверяет ту же ссылку для объектов
- Пример: `total === 25000`

toEqual()

- Рекурсивно сравнивает содержимое
- Подходит для объектов и массивов
- Не требует одной ссылки
- Пример: `Product DTO`

Проверка объекта Product

JAVASCRIPT

```
expect(product).toEqual({  
  id: 15,  
  name: "Notebook",  
  price: 420000,  
  inStock: true  
});
```

Что важно

- toEqual() сравнивает значения полей объекта.
- Удобно проверять DTO и результаты Service.

Проверка ошибок

- `toThrow()` используется для `sync functions`.
- Для `async rejection` используется `rejects`.
- Negative test должен падать, если ожидаемая ошибка не произошла.
- Проверять можно тип ошибки или часть `message`.

toThrow()

JAVASCRIPT

```
expect(() =>
  calculateOrderTotal([
    { price: 10000, quantity: 0 }
  ])
).toThrow(/quantity/);
```

Jest assertions: **value** → **expect()** → **matcher** → **PASS/FAIL**

1. Проверяемое значение

Значение или результат выполнения функции

42 число
"text" строка
[1, 2, 3] массив
{ ... } объект
f() результат функции

2. expect()

Оборачивает (принимает) проверяемое значение

expect(value)

expect() получает значение и возвращает объект, у которого есть матчеры (методы проверки).

3. matcher

Задаёт, какое условие нужно проверить

.toBe()
.toEqual()
.toThrow()
.toContain()
и другие матчеры...

4. Результат проверки

Тест либо проходит, либо падает



PASS

Условие выполнено.
Тест проходит.



FAIL

Условие не выполнено.
Тест падает с ошибкой.

1. toBe

сравнение примитивов
(точное равенство)

=

Проверяет, что значение строго равно ожидаемому (для чисел, строк, булевых и т. д.).

2. toEqual

сравнение объектов
и массивов

{ }

Глубокое сравнение.
Проверяет, что объекты или массивы имеют одинаковую структуру и значения.

3. toThrow

ожидаемая ошибка



Проверяет, что функция выбрасывает (генерирует) ошибку указанного типа или с нужным сообщением.

4. toContain

наличие элемента
в массиве или строке



Проверяет, что массив или строка содержит указанное значение (элемент или подстроку).

JS

```
const total = 25000;  
expect(total).toBe(25000);
```

JS

```
const product = { id: 1,  
  name: "Mouse" };  
expect(product).toEqual({  
  id: 1, name: "Mouse" });
```

JS

```
expect(() =>  
  calculateOrderTotal(null)  
) .toThrow(TypeError);
```

JS

```
const categories = [  
  "Laptops", "Phones", "Tablets"  
];  
expect(categories).toContain("Laptops");
```



Учебный вопрос 3. Setup, teardown и async tests

Tests должны начинаться с предсказуемого состояния

beforeEach() и afterEach()

- beforeEach выполняется перед каждым test в scope.
- afterEach выполняется после каждого test.
- Подходит для сброса mock state и test data.
- Каждый test получает одинаковую стартовую точку.
- Setup не должен скрывать важные данные конкретного scenario.

beforeEach()

JAVASCRIPT

```
let productRepository;  
  
beforeEach(() => {  
  productRepository = {  
    findById: jest.fn(),  
    create: jest.fn()  
  };  
});
```

beforeAll() и afterAll()

- beforeAll выполняется один раз перед tests текущего scope.
- afterAll выполняется один раз после них.
- Полезно для дорогого общего resource.
- Для unit mocks чаще безопаснее beforeEach.
- В integration tests hooks могут поднимать и очищать test database.

Async test через async/await

JAVASCRIPT

```
test("returns product", async () => {  
  const product = await service.getById(15);  
  
  expect(product.id).toBe(15);  
});
```

Что важно

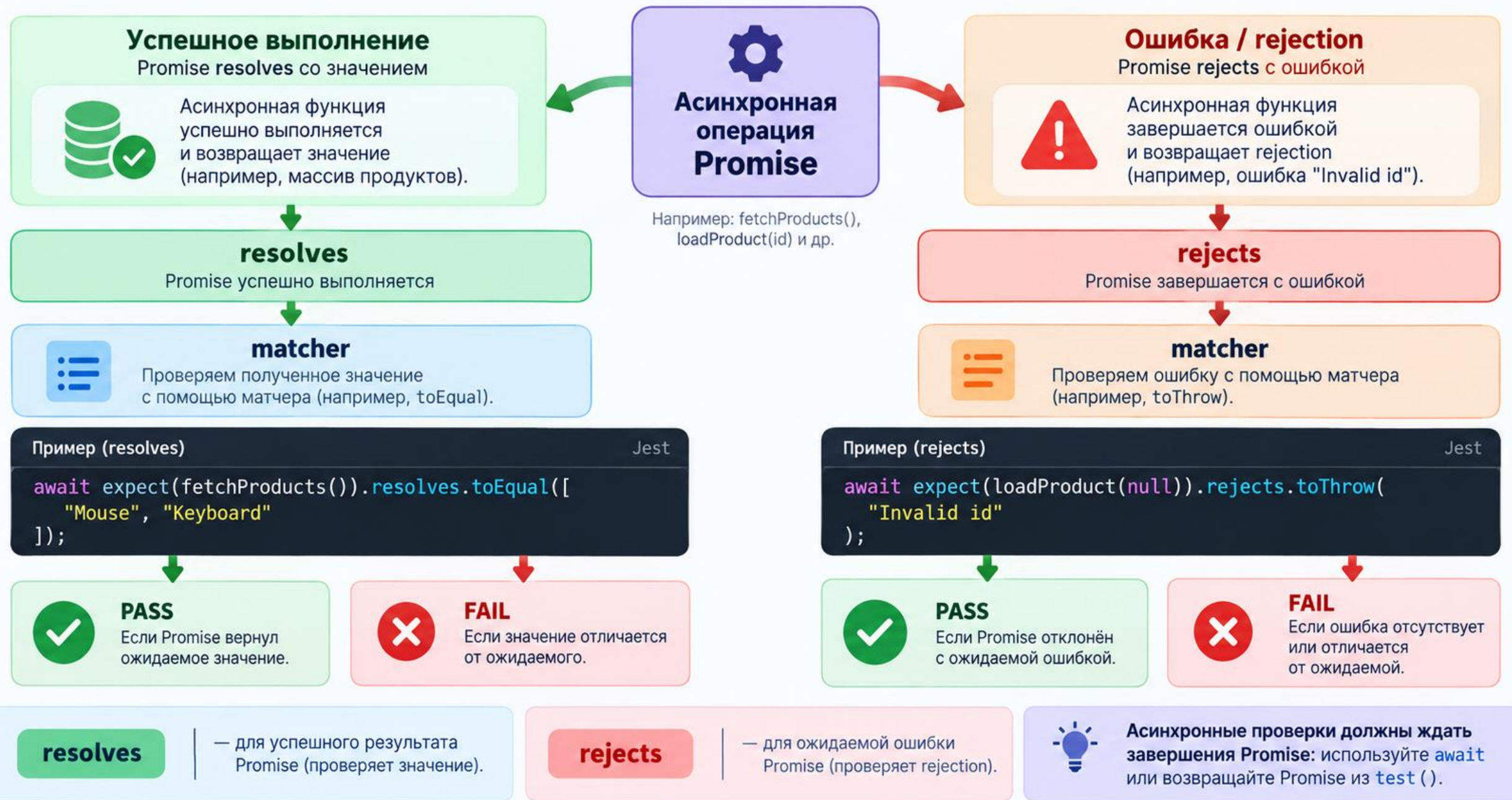
- Jest ждёт Promise, возвращаемый async test.
- Assertion выполняется после await.
- Async error делает test failed.

resolves и rejects

JAVASCRIPT

```
await expect(service.getById(15))  
  .resolves.toEqual(product);  
  
await expect(service.getById(999))  
  .rejects.toThrow("Product not found");
```

Async Jest test: Promise → resolves или rejects → matcher





Учебный вопрос 4. Mock functions

Mock заменяет dependency управляемой функцией для unit test

jest.fn()

- `jest.fn()` создаёт mock function, которая запоминает вызовы и может возвращать заданные test values.
- Можно задать return value.
- Можно задать resolved/rejected Promise.
- Mock записывает arguments каждого вызова.
- Можно проверять число вызовов.
- Это удобно для repository, mail service и внешнего API.

Mock return values

JAVASCRIPT

```
const findById = jest.fn();

findById.mockResolvedValue({
  id: 15,
  name: "Notebook"
});
```

Основные mock helpers

API	Назначение
<code>jest.fn()</code>	Создать mock function
<code>mockReturnValue()</code>	Вернуть sync value
<code>mockResolvedValue()</code>	Вернуть resolved Promise
<code>mockRejectedValue()</code>	Вернуть rejected Promise
<code>toHaveBeenCalled()</code>	Проверить факт вызова
<code>toHaveBeenCalledWith()</code>	Проверить arguments
<code>toHaveBeenCalledTimes()</code>	Проверить число вызовов

Проверка взаимодействия

JAVASCRIPT

```
expect(productRepository.findById)  
  .toHaveBeenCalledWith(15);
```

```
expect(productRepository.findById)  
  .toHaveBeenCalledTimes(1);
```

Что важно

- Проверяем не только результат Service.
- Проверяем, как Service взаимодействовал с dependency.
- Это полезно для orchestration logic.

Spy

- Spy позволяет наблюдать за вызовами существующего метода и при необходимости временно изменить его реализацию.
- В Jest используется `jest.spyOn(object, methodName)`.
- Spy полезен, когда объект уже существует.
- После `test original behavior` нужно восстановить.
- Для нового fake dependency часто проще использовать `jest.fn()`.

jest.spyOn()

JAVASCRIPT

```
const spy = jest.spyOn(logger, "info")
  .mockImplementation(() => {});

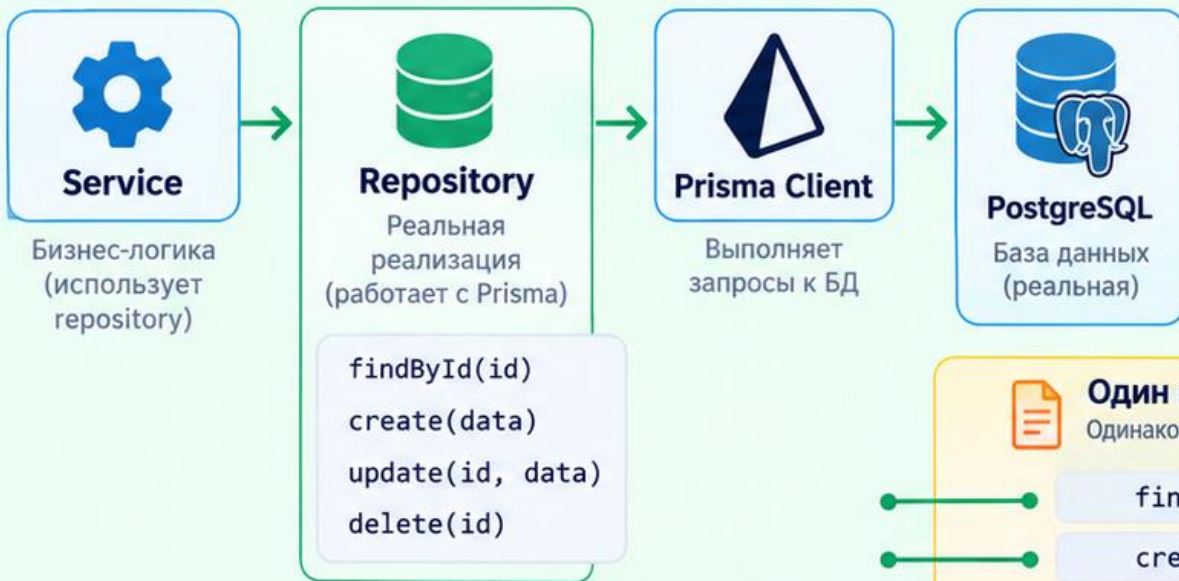
// ... test ...

expect(spy).toHaveBeenCalled();
spy.mockRestore();
```

Real dependency vs jest.fn mock: Service видит тот же contract

Production

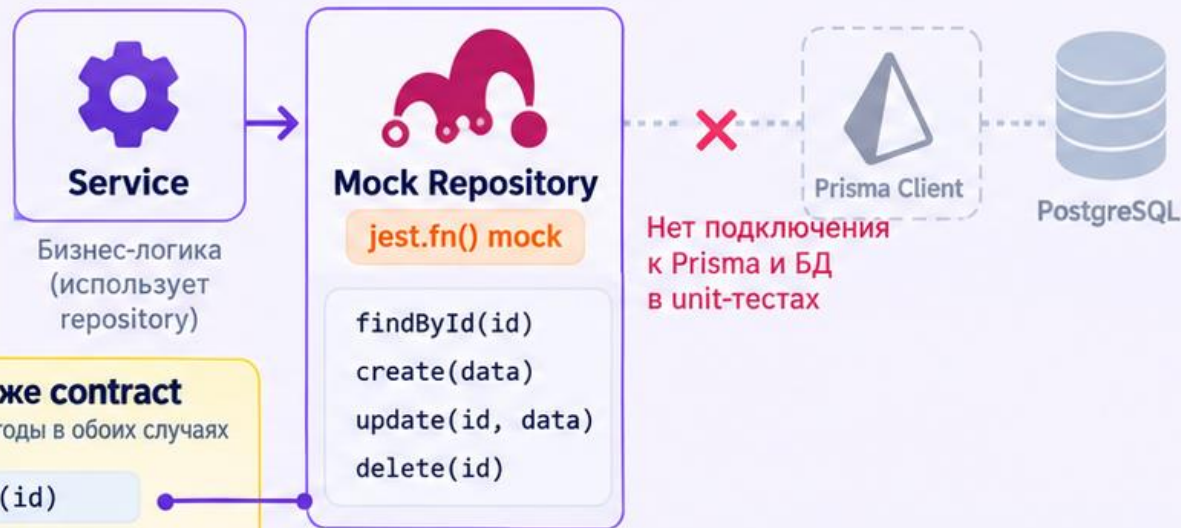
Реальные зависимости в работающем приложении



```
// Production (реальное приложение)  
const service = createProductService(productRepository);
```

Unit test

В unit-тестах сервис получает mock вместо реального repository



Один и тот же contract

Одинаковые методы в обоих случаях

findById(id)

create(data)

update(id, data)

delete(id)

```
findById(id)  
create(data)  
update(id, data)  
delete(id)
```

// Unit test (с использованием mock)

```
const mockRepo = {  
  findById: jest.fn(),  
  create: jest.fn(),  
};  
const service = createProductService(mockRepo);
```



Service видит тот же contract

Сервис зависит от интерфейса repository, а не от конкретной реализации.

1

Production: реальный repository работает с Prisma/PostgreSQL

2

Test: mock repository имитирует ответы через jest.fn()

3

Service тестируется изолированно, без запуска базы данных



Учебный вопрос 5. Unit testing Service layer

Service тестируем отдельно от Prisma и PostgreSQL

Проблема прямого импорта Prisma

- Если Service напрямую импортирует singleton PrismaClient, unit test сложнее изолировать.
- Запуск реальной DB превращает проверку в integration test.
- ESM module mocking в Jest остаётся experimental.
- Для учебного unit testing удобнее явно передать repository dependency.
- Это делает архитектуру тестируемой без реальной БД.

Product Service через dependency injection

JAVASCRIPT

```
export function createProductService(productRepository) {  
  return {  
    async getById(id) {  
      const product = await productRepository.findById(id);  
  
      if (!product) {  
        throw new Error("Product not found");  
      }  
  
      return product;  
    }  
  };  
}
```

Что важно

- Service зависит от contract repository, а не от конкретного PrismaClient.
- Production передаст Prisma repository.
- Unit test передаст mock repository.

Unit test Product Service

JAVASCRIPT

```
test("returns product by id", async () => {
  const repository = {
    findById: jest.fn().mockResolvedValue({
      id: 15,
      name: "Notebook"
    })
  };

  const service = createProductService(repository);
  const product = await service.getById(15);

  expect(product).toEqual({
    id: 15,
    name: "Notebook"
  });
  expect(repository.findById)
    .toHaveBeenCalledWith(15);
});
```

Что важно

- PostgreSQL не запускается.
- Mock задаёт контролируемый результат repository.
- Проверяется output и interaction.

Negative Service test

JAVASCRIPT

```
test("throws when product does not exist", async () => {  
  const repository = {  
    findById: jest.fn().mockResolvedValue(null)  
  };  
  
  const service = createProductService(repository);  
  
  await expect(service.getById(999))  
    .rejects.toThrow("Product not found");  
});
```

Unit Service test: Jest → mock repository → Service

Тестируем сервис в изоляции. Используем mock вместо реальной базы данных.



Проверяем логику сервиса,
а не работу базы данных.

Запуск unit теста (Prisma и PostgreSQL не запускаются)

1

Mock repository

Создаём mock с нужным поведением

```
// productRepository.mock.js
```

```
export const mockProductRepo = {
  findById: jest.fn()
    .mockResolvedValue({
      id: 1,
      name: 'Laptop',
      price: 1000
    }),
  create: jest.fn(),
  update: jest.fn(),
  delete: jest.fn()
};
```

jest.fn() создаёт функции-заглушки.
mockResolvedValue() задаёт,
что метод вернёт Promise с данными.

2

Service

Сервис использует тот же contract

```
// productService.js
```

```
export class ProductService {
  constructor(productRepo) {
    this.repo = productRepo; //
  }

  async getById(id) {
    return this.repo.findById(id);
  }
}
```

Сервис не знает, реальный это репозиторий или mock. Он просто вызывает методы по контракту.

3

Вызов сервиса

Вызываем метод, получаем результат

```
// productService.test.js
```

```
import { ProductService } from
  '../src/services/productService.js';
import { mockProductRepo } from
  './mocks/productRepository.mock.js';

test('getById returns product',
  async () => {
    const service = new ProductService(
      mockProductRepo);

    const result = await service.getById(1);

    expect(result).toEqual({
      id: 1,
      name: 'Laptop',
      price: 1000
    });

    expect(mockProductRepo.findById)
      .toHaveBeenCalledWith(1);
  });
```

4

Проверка (expect)

Проверяем результат и вызовы методов

```
// assertions
```

```
expect(result).toEqual({
  id: 1,
  name: 'Laptop',
  price: 1000
});

expect(mockProductRepo.findById)
  .toHaveBeenCalledWith(1);

expect(mockProductRepo.findById)
  .toHaveBeenCalledTimes(1);
```



PASS

Тест пройден.
Логика сервиса работает
корректно.

Вне unit теста

Prisma и PostgreSQL
не запускаются



- ✗ Нет подключения к БД
- ✗ Нет миграций
- ✗ Нет реальных данных
- ✗ Тесты быстрые и стабильные
- ✗ Тестируем только логику сервиса



Быстрые тесты

Не нужно запускать базу данных.



Изолированная логика

Тестируем только сервис.



Тот же контракт

Сервис работает с mock так же,
как с реальным репозиторием.

"Unit test проверяет поведение,
а не инфраструктуру."

Что именно проверяет такой unit test

- Service передаёт правильный id в repository.
- Service возвращает найденный Product.
- Service формирует ожидаемую ошибку при null.
- Поведение не зависит от состояния PostgreSQL.
- Test остаётся быстрым и deterministic.

Checklist Jest unit test



Проверяет одно конкретное поведение



Имеет понятное имя



Использует describe/test



Использует expect + подходящий matcher



Dependency заменена mock там, где нужна изоляция



Mock state сбрасывается между tests



Async Promise ожидается через await



Есть positive и negative scenarios



Не используется production DB



Test запускается через npm test

Контрольные вопросы

1

1. Для чего используется `describe()`?

2

2. Что делает `expect()`?

3

3. Чем `toBe()` отличается от `toEqual()`?

4

4. Для чего нужен `beforeEach()`?

5

5. Как проверить `rejected Promise`?

6

6. Что создаёт `jest.fn()`?

7

7. Что проверяет `toHaveBeenCalled()`?

8

8. Чем `mock` отличается от реальной `dependency`?

9

9. Почему `unit test Service` не должен запускать `PostgreSQL`?

10

10. Как `dependency injection` упрощает тестирование `Service`?

Итоги занятия

1

Jest объединяет test runner, assertions и mocking.

2

В ESM-проекте Jest запускаем через Node с `--experimental-vm-modules`.

3

`describe()` группирует связанные tests.

4

`expect()` и `matchers` описывают expected behavior.

5

`beforeEach/afterEach` помогают поддерживать isolation.

6

Async functions тестируются через `async/await`, `resolves` и `rejects`.

7

`jest.fn()` создаёт управляемую mock function.

8

Mocks позволяют проверять calls и arguments.

9

Dependency injection позволяет тестировать Service без Prisma/PostgreSQL.

10

Unit tests остаются быстрыми, изолированными и deterministic.

Следующая лекция

- Интеграционное тестирование REST API с Supertest
- Express app без app.listen()
- Supertest request agent
- GET/POST/PATCH/DELETE tests
- HTTP status и JSON assertions
- Auth headers
- Test database
- Setup/cleanup данных